1. Trabajando con ramas en local

Algunas personas resaltan que uno de los puntos más fuertes de Git es su sistema de ramificaciones y lo cierto es que esto le hace resaltar sobre los otros sistemas de control de versiones. ¿Por qué esto es tan importante? La forma en la que Git maneja las ramificaciones es increíblemente rápida, haciendo así de las operaciones de ramificación algo casi instantáneo, al igual que el avance o el retroceso entre distintas ramas, lo cual también es tremendamente rápido. A diferencia de otros sistemas de control de versiones, Git promueve un ciclo de desarrollo donde las ramas se crean y se unen ramas entre sí, incluso varias veces en el mismo día. Entender y manejar esta opción te proporciona una poderosa y exclusiva herramienta que puede, literalmente, cambiar la forma en la que desarrollas.

Las ramas son caminos que puede tomar el desarrollo de un software, algo que ocurre naturalmente para resolver problemas o crear nuevas funcionalidades. En la práctica permiten que nuestro proyecto pueda tener diversos estados y que los desarrolladores sean capaces de pasar de uno a otro de una manera ágil.

Ramas podrás usar en muchas situaciones. Por ejemplo imagina que estás trabajando en un proyecto y quieres implementar una nueva funcionalidad en la que sabes que quizás tengas que invertir varios días. Posiblemente sea algo experimental, que no sabes si llegarás a incorporar, o bien es algo que tienes claro que vas a querer completar, pero que, dado que te va a ocupar un tiempo indeterminado, es posible que en medio de tu trabajo tengas que tocar tu código, en el estado en el que lo tienes en producción.

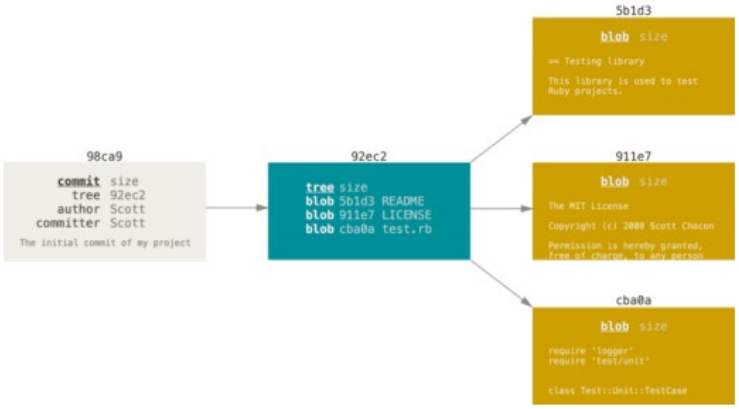
Bajo el supuesto anterior lo que haces es crear una rama. Trabajas dentro de esa rama por un tiempo, pero de repente se te cuelga el servidor y te das cuenta que hay cosas en el proyecto que no están funcionando correctamente. Los cambios de la rama no están completos, así que no los puedes subir. En ese instante, lo que las ramas Git te permiten es que, lanzando un sencillo comando, poner de nuevo el proyecto en el estado original que tenías, antes de empezar a hacer esos cambios que no has terminado. Perfecto! solucionas la incidencia, sobre el proyecto original y luego puedes volver a la rama experimental para seguir trabajando en esa idea nueva.
Llegará un momento en el que, quizás, aquellos cambios experimentales los quieras subir a producción. Entonces harás un proceso de fusionado entre la rama experimental y la rama original, operación que se conoce como merge en Git.

¿Cómo almacena git las instantáneas?

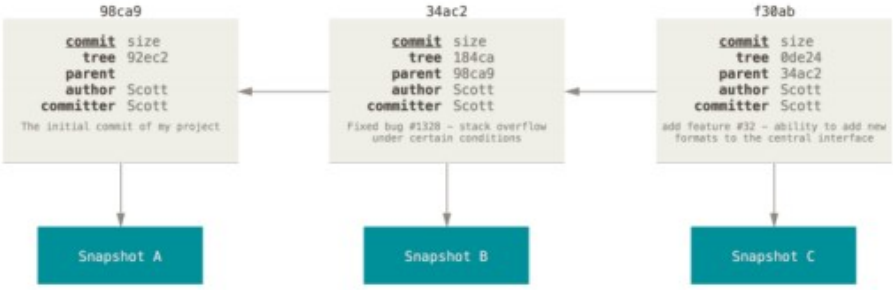
Al preparar los archivos Git almacena una copia de cada uno en el repositorio (estas copias se denominan "blobs"),y guarda la suma de control de cada fichero en el área de preparación

Cuando creas una confirmación con el comando git commit Git realiza sumas de control de cada subdirectorio y las guarda como objetos árbol en el repositorio Git. Después, Git crea un objeto de confirmación con los metadatos pertinentes y un apuntador al objeto árbol raíz del proyecto.

Para ilustrar esto, vamos a suponer, por ejemplo, que tienes una carpeta con tres archivos, que preparas (stage) todos ellos y los confirmas (commit).



Si haces más cambios y vuelves a confirmar, la siguiente confirmación guardará un apuntador su confirmación precedente.

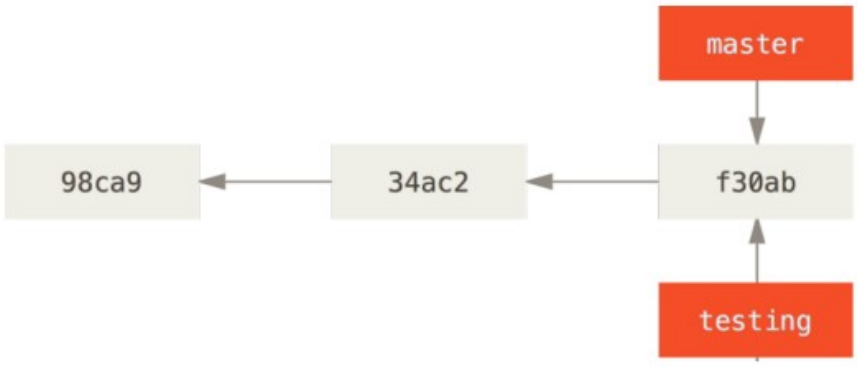


Una rama Git es simplemente un apuntador móvil apuntando a una de esas confirmaciones. La rama por defecto de Git es la rama master. Con la primera confirmación de cambios que realicemos, se creará esta rama principal master **(actualmente se ha modificado por main)** apuntando a dicha confirmación. En cada confirmación de cambios que realicemos, la rama irá avanzando automáticamente.

Creando una nueva rama

¿Qué sucede cuando creas una nueva rama? Bueno…, simplemente se crea un nuevo apuntador para que lo puedas mover libremente. Por ejemplo, supongamos que quieres crear una rama nueva denominada "testing".

```
git branch testing
```



Y, ¿cómo sabe Git en qué rama estás en este momento? Pues…, mediante un apuntador especial denominado HEAD. En Git, es simplemente el apuntador a la rama local en la que tú estés en ese momento, en este caso la rama master; pues el comando git branch solamente crea una nueva rama, y no salta a dicha rama.

Esto puedes verlo fácilmente al ejecutar el comando git log para que te muestre a dónde apunta cada rama. Esta opción se llama --decorate

```
git log --oneline -decorate
```

Cambiar de rama

Para saltar de una rama a otra, tienes que utilizar el comando git checkout. Hagamos una prueba, saltando a la rama testing recién creada. Esto mueve el apuntador HEAD a la rama testing.

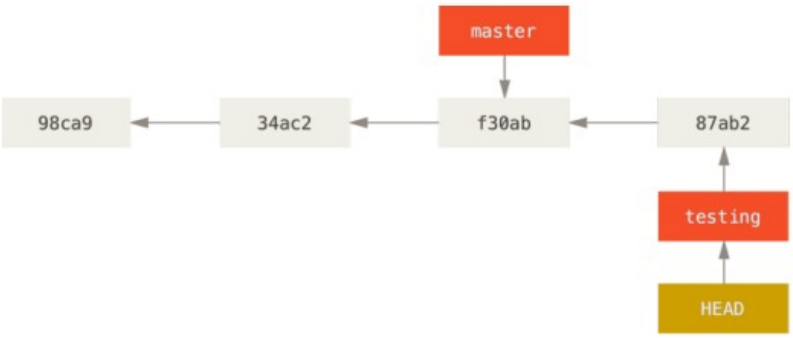
```
git checkout testing
```

Para crear una nueva rama y saltar a ella, en un solo paso, puedes utilizar el comando git checkout con la opción -b

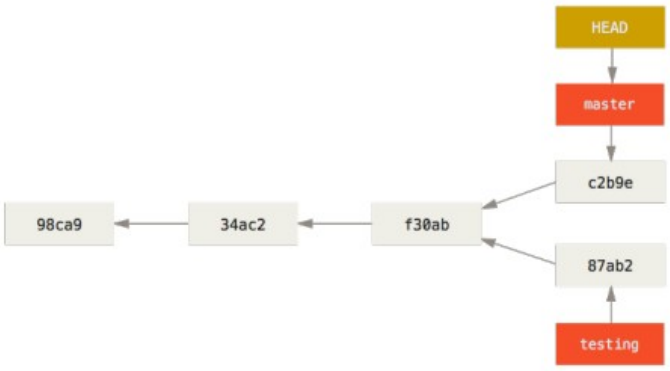
Si hacemos cambios…

```
$ nano test.rb

$ git commit -a -m 'made a change'
```



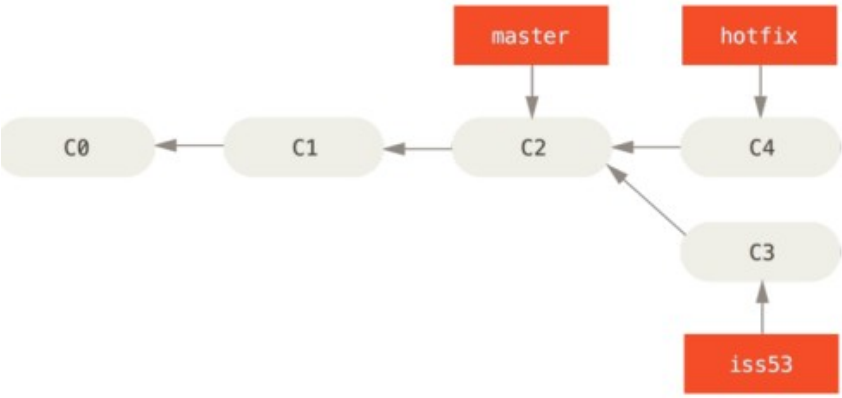
Si vuelves a la rama master y haces cambios…



Si ejecutas git log --oneline --decorate --graph --all te mostrará el historial de tus confirmaciones, indicando dónde están los apuntadores de tus ramas y como ha divergido tu historial.

Combinando ramas

Partimos del siguiente escenario, estando en máster estas trabajando en un proyecto y tienes un par de confirmaciones (commit) ya realizadas. Decides trabajar en el problema #53 y para ello creas una nueva rama iss53, trabajas en el sitio web y haces algunas confirmaciones de cambios (commits). Entonces, recibes una llamada avisándote de otro problema urgente en el sitio web y debes resolverlo inmediatamente. Al usar Git, no necesitas mezclar el nuevo problema con los cambios que ya habías realizado sobre el problema #53; ni tampoco perder tiempo revirtiendo esos cambios para poder trabajar sobre el contenido que está en producción. Basta con saltar de nuevo a la rama master y continuar trabajando a partir de allí. Vamos a crear una nueva rama hotfix, sobre la que trabajar hasta resolverlo, confirmamos con commit (C4)

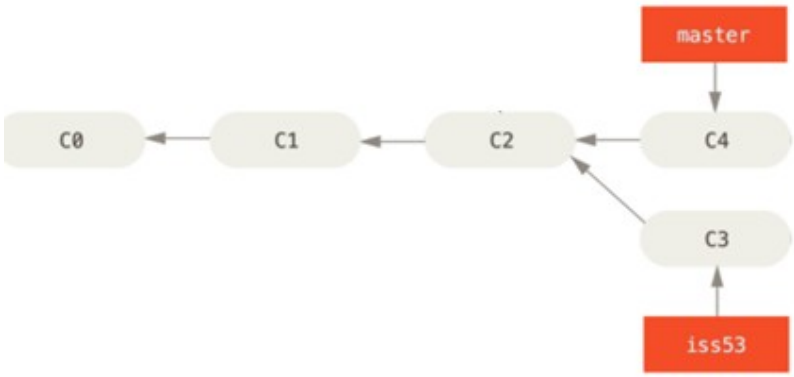


Ahora quieres incorporar los cambios de la rama hotfix a la rama master para ponerlos en producción. Esto se hace con el comando git merge.

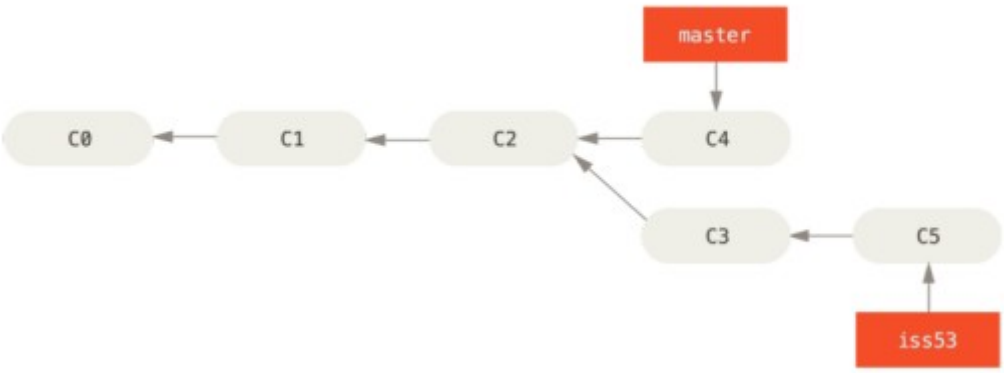
```
$ git checkout master
$ git merge hotfix
Updating f42c576..3a0874c
Fast-forward
 index.html | 2 ++
 1 file changed, 2 insertions(+)
```

En este caso se hará un “Fast forward” (“Avance rápido”, en inglés) que aparece en la salida del comando. Git ha movido el apuntador hacia adelante, ya que la confirmación apuntada en la rama donde has fusionado estaba directamente arriba respecto a la confirmación actual. Dicho de otro modo: cuando intentas fusionar una confirmación con otra confirmación accesible siguiendo directamente el historial de la primera; Git simplifica las cosas avanzando el puntero, ya que no hay ningún otro trabajo divergente a fusionar. Esto es lo que se denomina “avance rápido” (“fast forward”). Ahora, los cambios realizados están ya en la instantánea (snapshot) de la confirmación (commit) apuntada por la rama master y se puede borrar la rama hotfix. Para borrar

```
git branch -d hotfix
```



Tras haber resuelto el problema urgente que había interrumpido tu trabajo, puedes volver a donde estabas y seguir haciendo commits…

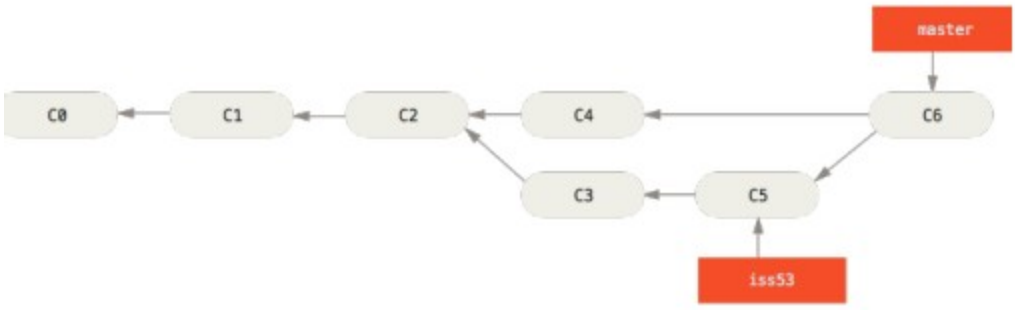


Cabe destacar que todo el trabajo realizado en la rama hotfix no está en los archivos de la rama iss53. Si fuera necesario agregarlos, puedes fusionar (merge) la rama master sobre la rama iss53 utilizando el comando git merge master, o puedes esperar hasta que decidas fusionar (merge) la rama iss53 a la rama master. Vamos a tomar esa decisión. Seguimos trabajando en la rama iss53 y una vez terminado fusionamos con la rama master.

```
$ git checkout master
Switched to branch 'master'
$ git merge iss53
Merge made by the 'recursive' strategy.
index.html |    1 +
1 file changed, 1 insertion(+)
```

Es algo diferente de la fusión realizada anteriormente con hotfix. En este caso, el registro de desarrollo había divergido en un punto anterior. Debido a que la confirmación en la rama actual no es ancestro directo de la rama que pretendes fusionar, Git tiene cierto trabajo extra que hacer. Git realizará una fusión a tres bandas, utilizando las dos instantáneas apuntadas por el extremo de cada una de las ramas y por el ancestro común a ambas.

En lugar de simplemente avanzar el apuntador de la rama, Git crea una nueva instantánea (snapshot) resultante de la fusión a tres bandas; y crea automáticamente una nueva confirmación de cambios (commit) que apunta a ella. Nos referimos a este proceso como "fusión confirmada" y su particularidad es que tiene más de un padre.



En algunas ocasiones, los procesos de fusión no suelen ser fluidos. Si hay modificaciones dispares en una misma porción de un mismo archivo en las dos ramas distintas que pretendes fusionar, Git no será capaz de fusionarlas directamente. Git no crea automáticamente una nueva fusión confirmada (merge commit), sino que hace una pausa en el proceso, esperando a que tú resuelvas el conflicto. Todo aquello que sea conflictivo y no se haya podido resolver, se marca como "sin fusionar" (unmerged). Git añade a los archivos conflictivos unos marcadores especiales de

resolución de conflictos que te guiarán cuando abras manualmente los archivos implicados y los edites para corregirlos. El archivo conflictivo contendrá algo como:

```
<<<<<<< HEAD:index.html
<div id="footer">contact : email.support@github.com</div>
=======
<div id="footer">
 please contact us at support@github.com
</div>
>>>>>>> iss53:index.html
```

Donde nos dice que la versión en HEAD (la rama master, la que habías activado antes de lanzar el comando de fusión) contiene lo indicado en la parte superior del bloque (todo lo que está encima de =======) y que la versión en iss53 contiene el resto, lo indicado en la parte inferior del bloque. Para resolver el conflicto, has de elegir manualmente el contenido de uno o de otro lado.

Tras resolver todos los bloques conflictivos, has de lanzar comandos git add para marcar cada archivo modificado. Marcar archivos como preparados (staged) indica a Git que sus conflictos han sido resueltos.

2. PRÁCTICA2